PetPal Final Report

CG2271 Assignment 4

Group B01_14

17 April 2026

1. Introduction
2. Sensors, Actuators, and Data Handling
   2.1 Sensors
   2.2 Actuators
   2.3 Libraries and Frameworks Used
3. Hardware Architecture
   3.1 System-Level Layout
   3.2 ESP32-S2 Pin Table
   3.3 MCXC444 Pin Table
4. Software Architecture
   4.1 ESP32-S2 Firmware
   4.2 MCXC444 Firmware
   4.3 Finite State Behaviour
5. Coordination and Synchronization
   5.1 Interrupt Service Routines
   5.2 Semaphore
   5.3 Queue
   5.4 Mutex
6. Communication Protocol Between ESP32-S2 and MCXC444
   6.1 MCXC444 to ESP32-S2
   6.2 ESP32-S2 to MCXC444
7. Online Function: Firebase Realtime Database
   7.1 Data Sent by ESP32
   7.2 Data Received by ESP32
   7.3 Dashboard Role
8. Special Features
   8.1 Dual-Board Responsibility Split
   8.2 Cloud-Controlled Physical Actions
   8.3 Sensor Fusion for Pet Presence
   8.4 Local and Remote Feedback
9. Challenges Faced and Solutions
   9.1 Faulty Vibration Sensor
   9.2 Coordinating Two Microcontrollers
   9.3 Avoiding ISR Overload
   9.4 Cloud Integration on a Microcontroller
   9.5 Dashboard Data Path
10. Requirement Mapping

11. Conclusion

1. Introduction

PetPal is an Internet-connected pet care device built using the ESP32-S2 and MCXC444 microcontrollers. The product objective is to help a pet owner monitor the pet's immediate environment, detect when the pet is nearby, and remotely trigger simple care or play actions through a web dashboard. The system combines local sensing, local actuation, inter-board communication, FreeRTOS task design, and Firebase Realtime Database as the cloud-based service.

The problem addressed by PetPal is that owners may not always be near their pet's food, water, and play area. A small embedded device can provide useful awareness by reporting temperature, humidity, water level, pet presence, and interaction events to a dashboard. The same dashboard can also issue commands such as dispensing food or starting play mode. PetPal therefore acts as a simple smart pet station rather than a collection of unrelated sensors.

Compared with the interim version, the final design was updated in one important hardware area. The original vibration sensor was faulty, so it was replaced by a GY-521 module, which contains the MPU6050 accelerometer and gyroscope. In the final implementation, the GY-521 is used as an interrupt-driven motion or shock sensor. This preserves the intended purpose of detecting sudden pet interaction, while improving reliability and allowing the system to satisfy the interrupt-driven sensor requirement.

The final feature list is:

- Pet presence detection using the HC-SR04 ultrasonic sensor on the MCXC444.
- Shock or motion detection using the GY-521 MPU6050 interrupt output on the ESP32-S2.
- Temperature and humidity monitoring using a DHT11 sensor on the ESP32-S2.
- Water level monitoring using an analog water level sensor on the ESP32-S2.
- Food dispensing using a servo motor on the MCXC444.
- Play mode using a laser emitter controlled by the ESP32-S2 and a sweep servo on the MCXC444.
- Local feedback through an onboard LED on the MCXC444 and a passive buzzer on the ESP32-S2.
- Bidirectional UART communication between ESP32-S2 and MCXC444.
- Firebase Realtime Database integration for cloud telemetry and dashboard commands.
- A web dashboard that displays live telemetry and sends feed/play commands.

2. Sensors, Actuators, and Data Handling

2.1 Sensors

Sensor	Board	Interface	Data Produced	Handling in Software
HC-SR04 ultrasonic sensor	MCXC444	GPIO trigger + GPIO echo interrupt	Distance in centimetres	MCXC444 measures echo pulse width using TPM1 and sends distance to ESP32 over UART.
GY-521 MPU6050	ESP32-S2	I2C + interrupt pin	Motion/shock event	ESP32 configures MPU6050 motion detection and handles the interrupt on GPIO3.
DHT11	ESP32-S2	Digital single-wire protocol	Temperature and humidity	ESP32 polls every 2 seconds using the Adafruit DHT library.
Water level sensor	ESP32-S2	ADC	Raw analog level	ESP32 polls every 500 ms and maps the reading to empty, low, ok, or full.

The ultrasonic sensor is used for distance-based pet presence. The MCXC444 emits a short trigger pulse, captures rising and falling echo edges through an ISR, and converts the pulse duration into distance. The ESP32 applies hysteresis with `PET_NEAR_CM` and `PET_FAR_CM` thresholds so the pet state does not rapidly flicker when the pet is near the boundary.

The GY-521 replaces the faulty vibration sensor. The ESP32 communicates with it through I2C and enables the MPU6050 motion interrupt. When the module detects movement, it asserts the interrupt pin. The ESP32 ISR records a shock event, applies debounce timing, and the main loop later publishes the event to Firebase. This makes the GY-521 the interrupt-driven sensor on the ESP32 side.

The DHT11 and water sensor are polled. Their values are included in the telemetry JSON sent to Firebase. Temperature and humidity are displayed directly on the dashboard. The water level raw value is also converted into a user-facing level label.

2.2 Actuators

Actuator	Board	Interface	Purpose
Food servo	MCXC444	TPM0 PWM channel 0	Opens and closes the food gate when a feed command is received.
Laser sweep servo	MCXC444	TPM0 PWM channel 1	Sweeps the laser direction during play mode.
Laser emitter	ESP32-S2	GPIO output	Turns the play laser on or off.
Passive buzzer	ESP32-S2	LEDC PWM	Provides audible feedback for shock, pet arrival, and feed events.
Onboard LED	MCXC444	GPIO output	Shows local pet detection status.

The actuators are tied to sensor and dashboard state. For example, the LED is not driven by a timer alone. It reflects the pet detection state received from the ESP32. The food servo responds to a cloud dashboard command after the ESP32 polls Firebase. The play mode uses both boards: the ESP32 controls the laser emitter while the MCXC444 sweeps the laser servo.

2.3 Libraries and Frameworks Used

The ESP32-S2 firmware uses:

- `WiFi.h` for WiFi connectivity.
- `WiFiClientSecure.h` and `HTTPClient.h` for HTTPS REST calls to Firebase.
- `DHT.h` for DHT11 temperature and humidity.
- `Wire.h` for I2C.
- `MPU6050.h` for GY-521/MPU6050 configuration.
- Arduino `Serial1` for UART communication with the MCXC444.

The MCXC444 firmware uses:

- NXP SDK drivers such as `fsl_gpio.h`, `fsl_port.h`, `fsl_tpm.h`, `fsl_uart.h`, and `fsl_debug_console.h`.
- FreeRTOS headers `FreeRTOS.h`, `task.h`, `queue.h`, and `semphr.h`.

The dashboard uses:

- Next.js and React for the web interface.
- Firebase Realtime Database REST calls in the Next.js API routes.
- Environment variables for Firebase configuration.

3. Hardware Architecture

3.1 System-Level Layout

The final system uses the ESP32-S2 as the Internet gateway and dashboard bridge, while the MCXC444 handles low-level timing-sensitive hardware. This division is useful because the ESP32-S2 has WiFi support and mature HTTPS libraries, while the MCXC444 is well suited for deterministic GPIO interrupts, TPM timers, PWM, and FreeRTOS task scheduling.

```
            Firebase Realtime Database
                     ^
                     |
             HTTPS REST over WiFi
                     |
                ESP32-S2
DHT11, water, GY-521, buzzer, laser
                     ^
                     |
              UART 115200 bps
                     |
                MCXC444
HC-SR04, food servo, laser servo, LED
```

3.2 ESP32-S2 Pin Table

ESP32-S2 Pin	Connected Component	Direction	Notes
GPIO1	Water level sensor AO	Input	ADC reading for water level.
GPIO3	GY-521 MPU6050 INT	Input	Interrupt-driven shock/motion detection.
GPIO7	Passive buzzer	Output	LEDC PWM tone output.
GPIO8	GY-521 SDA	I2C	I2C data line.
GPIO9	Laser emitter	Output	Digital laser control.
GPIO10	GY-521 SCL	I2C	I2C clock line.
GPIO11	DHT11 data	Input	Temperature and humidity.
GPIO17	UART TX to MCXC444 PTE23	Output	ESP32 sends commands to MCXC444.
GPIO18	UART RX from MCXC444	Input	ESP32 receives distance and

ESP32-S2 Pin	Connected Component	Direction	Notes
	PTE22		debug text.

3.3 MCXC444 Pin Table

MCXC444 Pin	Connected Component	Direction	Notes
PTD2	HC-SR04 Trig	Output	Generates ultrasonic trigger pulse.
PTD4	HC-SR04 Echo	Input	Interrupt on rising and falling edges.
PTC1	Food servo	PWM output	TPM0 channel 0, 50 Hz servo PWM.
PTC2	Laser sweep servo	PWM output	TPM0 channel 1, 50 Hz servo PWM.
PTD5	Onboard green LED	Output	Active-low LED for pet status.
PTE22	UART2 TX to ESP32 GPIO18	Output	Sends distance packet to ESP32.
PTE23	UART2 RX from ESP32 GPIO17	Input	Receives pet status and dashboard commands.

4. Software Architecture

4.1 ESP32-S2 Firmware

The ESP32-S2 runs an Arduino-style main loop. Although the ESP32 Arduino core itself runs on FreeRTOS internally, the final explicit FreeRTOS task structure is implemented on the MCXC444. The ESP32 firmware is responsible for five major roles:

1. Connecting to WiFi and syncing time using NTP.
2. Sending telemetry to Firebase Realtime Database.
3. Polling Firebase for dashboard commands.
4. Bridging commands and pet status to the MCXC444 over UART.
5. Reading local ESP32-side sensors and controlling local actuators.

The main loop is non-blocking except for short delays and timed operations. It uses `millis()` to schedule water sensor reads, DHT11 reads, telemetry uploads, command polling, WiFi reconnection, and serial status prints. This prevents one activity, such as Firebase polling, from completely stopping sensor handling.

The ESP32 telemetry object includes:

- `deviceId`
- `mode`
- `uptimeSec`
- `temperatureC`
- `humidityPct`
- `distanceCm`
- `shockDetected`
- `petAround`
- `lastTriggerSensor`
- `lastSeenAt`
- `presenceUpdatedAt`
- `waterLevelRaw`
- `waterLevel`
- `laserOn`
- `playServoMoving`
- `feederTriggered`
- `buzzerOn`
- `lastEvent`
- `lastFeedTs`
- `lastPlayTs`
- `updatedAt`
- `updatedAtMs`

The ESP32 command polling logic reads a single command object from Firebase. If its status is `queued`, the ESP32 parses its `type`. A `feed_now` command stops play mode if necessary, sends a feed command to the MCXC444, sounds the buzzer, and records the feed event. A `play_mode_toggle` command either starts or stops play mode by controlling the local laser and sending a play or stop command to the MCXC444.

4.2 MCXC444 Firmware

The MCXC444 firmware is structured around FreeRTOS. Its main function initializes pins, timers, UART2, interrupts, queues, semaphores, and tasks. The scheduler is then started with `vTaskStartScheduler()`.

The MCXC444 has three main tasks:

Task	Responsibility	Timing/Synchronization
`sensor_task`	Triggers HC-SR04, waits for echo result, sends distance to ESP32.	Uses `echoSemaphore` signalled by echo ISR. Runs about every 140 ms.
`command_task`	Parses UART bytes from ESP32 into system commands and pet status.	Receives bytes from `cmdQueue`, which is filled by UART RX ISR.
`actuator_task`	Drives LED, food servo, and laser sweep servo based on shared mode state.	Reads shared state using `stateMutex`.

This task split separates sensing, communication, and actuation. The ultrasonic timing is isolated from command parsing, and actuator movement is not performed directly inside interrupts.

4.3 Finite State Behaviour

The MCXC444 maintains a simple system mode:

State	Meaning	Actuator Behaviour
`MODE_IDLE`	No active feeding or play command.	Food servo and laser servo return to centre.
`MODE_FEEDING`	Dashboard requested food dispense.	Food servo opens, waits, closes, then returns to idle.
`MODE_PLAYING`	Dashboard requested play mode.	Laser servo sweeps between left and right positions.

The ESP32 maintains an additional presence state. It combines ultrasonic distance and GY-521 shock events into a `petAround` field for the dashboard. A short hold time prevents brief sensor dropouts from making the dashboard flicker between present and absent.

5. Coordination and Synchronization

The project uses both interrupts and FreeRTOS synchronization primitives.

5.1 Interrupt Service Routines

The system contains two important MCXC444 ISRs:

- The ultrasonic echo ISR captures rising and falling edges on the echo pin. It records the timer count at the rising edge and calculates pulse width at the falling edge.
- The UART2 RX ISR reads incoming bytes from the ESP32 and pushes them into a FreeRTOS queue.

The ESP32 also uses an interrupt:

- The GY-521 MPU6050 interrupt line triggers `shockISR()`, which debounces the event and sets a flag for the main loop to process later.

The ISRs are intentionally short. They do not perform complex control logic, servo movement, or cloud operations. Instead, they record the event and notify normal code through a flag, semaphore, or queue.

5.2 Semaphore

The MCXC444 uses a binary semaphore named `echoSemaphore`. The ultrasonic echo ISR gives this semaphore when a valid echo pulse has been captured. The `sensor_task` takes the semaphore with a timeout. If the timeout expires, it reports a fallback distance of 999 cm to the ESP32. This prevents the task from blocking forever when no echo is received.

5.3 Queue

The MCXC444 uses `cmdQueue` to transfer UART bytes from the UART ISR to `command_task`. This is important because packet parsing is more complex than a simple byte read and should not run inside the ISR. The queue also protects against short bursts of incoming bytes.

5.4 Mutex

The MCXC444 uses `stateMutex` to protect shared state such as `currentMode` and `petDetected`. These values are written by `command_task` and read by `actuator_task`. Without the mutex, the actuator task could read inconsistent state while a command update is in progress.

6. Communication Protocol Between ESP32-S2 and MCXC444

Communication uses UART at 115200 bps. The protocol is binary and intentionally small so that it is easy to parse on both boards.

6.1 MCXC444 to ESP32-S2

The MCXC444 sends ultrasonic distance packets:

```
[0xAA][0x01][dist_hi][dist_lo]
```

`0xAA` is the packet header, `0x01` identifies a distance packet, and the final two bytes form a 16-bit distance value in centimetres. The ESP32 receives this packet on `Serial1`, reconstructs the distance, applies near/far hysteresis, and sends the current pet status back to the MCXC444.

6.2 ESP32-S2 to MCXC444

The ESP32 sends command packets:

```
[0xBB][type]
[0xBB][0x01][status]
```

The supported command types are:

Type	Meaning
`0x01`	Pet status update, followed by `0x00` or `0x01`.
`0x10`	Feed command.
`0x11`	Start play command.
`0x12`	Stop command.

This protocol provides bidirectional data exchange: MCXC444 sends sensor data to the ESP32, and the ESP32 sends both cloud commands and processed pet status back to the MCXC444.

7. Online Function: Firebase Realtime Database

The ESP32 connects to the Internet through WiFi and exchanges data with Firebase Realtime Database using HTTPS REST API calls. This satisfies the requirement that the ESP32 must connect to the Internet to exchange data with cloud-based services.

The database path used by the final implementation is:

```
/petpal/devices/esp32s2-petpal
```

Under this device path, two main child objects are used:

Firebase Path	Written By	Read By	Purpose
`/telemetry`	ESP32	Dashboard	Latest sensor readings, actuator status, presence state, and timestamps.
`/command`	Dashboard	ESP32	Latest queued dashboard command and execution status.

7.1 Data Sent by ESP32

The ESP32 periodically performs a Firebase `PUT` to:

```
/petpal/devices/esp32s2-petpal/telemetry
```

This upload includes sensor readings and derived state. For example, `distanceCm` comes from the MCXC444 ultrasonic sensor, `shockDetected` comes from the GY-521 interrupt, and `waterLevel` is derived from the raw ADC water level value. The dashboard uses this object to show temperature, humidity, water level, distance, pet presence, device status, and recent events.

7.2 Data Received by ESP32

The dashboard writes commands to:

```
/petpal/devices/esp32s2-petpal/command
```

The ESP32 polls this path. When it sees a `queued` command, it executes the command locally and forwards the appropriate binary command to the MCXC444 if servo action is needed. After execution, it updates the command status to `executed` or `failed`.

7.3 Dashboard Role

The dashboard does not communicate directly with the ESP32. Instead, Firebase acts as the shared cloud data layer:

```
ESP32-S2 -> Firebase -> Dashboard
Dashboard -> Firebase -> ESP32-S2 -> MCXC444
```

This architecture allows the dashboard to be hosted separately from the hardware. It also allows the ESP32 and web app to operate asynchronously. If the dashboard sends a command while the ESP32 is between polling cycles, the command remains in Firebase until the ESP32 reads it.

8. Special Features

8.1 Dual-Board Responsibility Split

The project uses each board for what it is best at. The ESP32-S2 handles WiFi, HTTPS, Firebase, and the browser-facing dashboard pipeline. The MCXC444 handles time-sensitive embedded control using FreeRTOS and hardware timers. This creates a cleaner architecture than forcing all logic onto one board.

8.2 Cloud-Controlled Physical Actions

The dashboard can trigger physical actions. A feed command becomes a Firebase command, then an ESP32 UART packet, then a MCXC444 FreeRTOS state change, and finally a servo movement. This demonstrates a complete cloud-to-hardware control path.

8.3 Sensor Fusion for Pet Presence

Pet presence is not based on one signal alone. The ultrasonic sensor detects distance, while the GY-521 detects motion or shock. The ESP32 publishes both raw indicators and a combined presence result. This improves the product coherence because multiple sensors contribute to a single user-facing feature: knowing whether the pet is around and interacting with PetPal.

8.4 Local and Remote Feedback

The system provides both local feedback and remote feedback. The MCXC444 LED reflects pet status locally. The ESP32 buzzer gives audible cues for events. The Firebase dashboard provides remote visibility and command control.

9. Challenges Faced and Solutions

9.1 Faulty Vibration Sensor

The original vibration sensor was unreliable, so it was replaced with a GY-521 MPU6050 module. The replacement required a change from a simple digital vibration input to an I2C-based motion sensor with an interrupt output. The solution was to initialize the MPU6050, enable motion detection, and attach an interrupt handler to the ESP32 GPIO connected to the module's INT pin. This preserved the intended interaction detection feature while improving reliability.

9.2 Coordinating Two Microcontrollers

The ESP32 and MCXC444 have different roles and toolchains, so the communication protocol had to be simple and robust. A compact binary UART protocol with fixed headers was used. The MCXC444 sends distance packets with header `0xAA`, while the ESP32 sends command packets with header `0xBB`. This makes it easy for each board to identify packet direction and type.

9.3 Avoiding ISR Overload

The ultrasonic echo and UART RX operations are interrupt-driven, but long work inside an ISR can make the system unstable. The solution was to keep ISRs short. The echo ISR only records timing and gives a semaphore. The UART ISR only reads a byte and sends it to a queue. Normal tasks handle the rest.

9.4 Cloud Integration on a Microcontroller

The ESP32 must connect to WiFi and use HTTPS requests to communicate with Firebase. This introduces possible failures such as WiFi timeout, TLS setup, and database permission issues. The firmware handles WiFi reconnection attempts, prints Firebase HTTP errors to the serial monitor, and uses Firebase REST paths that match the dashboard API routes.

9.5 Dashboard Data Path

An earlier dashboard version depended on a local Express API and simulator. The final version uses Firebase directly through Next.js API routes. This reduced the number of moving parts for the final hardware demo because the ESP32 and dashboard now share Firebase as the single cloud data layer.

10. Requirement Mapping

Requirement	Final Implementation
Coherent product	PetPal combines sensing, feeding, play, and dashboard monitoring for pet care.
At least 4 sensors for 4/5-member group	HC-SR04, GY-521, DHT11, water level sensor.
At least 3 actuators for 4/5-member group	Food servo, laser servo, laser emitter, buzzer, onboard LED.
At least 1 MCXC444 sensor	HC-SR04 ultrasonic sensor on MCXC444.
At least 1 polled sensor	DHT11 and water level sensor are polled by ESP32.
At least 1 interrupt-driven sensor	HC-SR04 echo interrupt on MCXC444 and GY-521 interrupt on ESP32.
At least 1 ISR and 3 tasks	MCXC444 has echo ISR, UART ISR, and three FreeRTOS tasks.
Mutex/semaphore and queue	`stateMutex`, `echoSemaphore`, and `cmdQueue` are used on MCXC444.
Bidirectional ESP32-MCXC444 data exchange	MCXC444 sends distance to ESP32; ESP32 sends pet status and commands to MCXC444.
ESP32 Internet/cloud exchange	ESP32 uses WiFi and Firebase Realtime Database REST API.
Online function	Web dashboard reads telemetry and writes feed/play commands through Firebase.

11. Conclusion

PetPal demonstrates a complete embedded IoT product using the ESP32-S2 and MCXC444. The ESP32-S2 acts as the cloud gateway and sensor hub for DHT11, water level, GY-521 shock detection, buzzer, and laser control. The MCXC444 handles ultrasonic sensing, servo control, LED output, and FreeRTOS task scheduling. The two boards exchange data through a compact UART protocol, and Firebase Realtime Database connects the embedded system to the dashboard.

The final system satisfies the project requirements by using multiple sensors and actuators, including both polled and interrupt-driven sensors, implementing FreeRTOS tasks and synchronization primitives, performing bidirectional inter-board communication, and exchanging data with a cloud-based service. The replacement of the faulty vibration sensor with the GY-521 was a meaningful design adjustment that improved reliability while preserving the original product purpose.

Overall, PetPal is a coherent smart pet care station: sensor data is not collected in isolation, and actuators are not driven arbitrarily. Instead, the sensors, dashboard, cloud database, and actuators work together to monitor a pet, report useful status, and allow simple remote care actions.